

TALLER 2

REDES NEURONALES

Sistemas Avanzados de Producción y Procesos

Ingeniería Industrial

Integrantes:

Omar Murcia 42750

Marcela Muñoz 119245

Paula Sanchez 121041

Introducción

El presente taller aborda el ciclo completo de un proyecto de analítica aplicada a la demanda de bicicletas compartidas en Washington D.C. (Capital Bikeshare, 2011–2012). El reto logístico central es claro: **predecir con precisión la demanda horaria de bicicletas** para evitar tanto el exceso de inventario —que genera costos de subutilización— como la escasez —que produce pérdida de clientes y fallos en la movilidad multimodal.

El dataset integra variables temporales (hora, día, mes, estación), ambientales (situación climática) y binarias (festivo, día laboral). A diferencia de la demanda de bienes convencionales, el alquiler de bicicletas presenta una alta sensibilidad a variables exógenas como el clima, lo que hace inviables los modelos lineales simples y motiva el uso de arquitecturas de redes neuronales capaces de capturar relaciones no lineales complejas.

Los objetivos técnicos del taller son: (1) derivar y fundamentar el estimador Ridge como herramienta de regularización matricial; (2) explorar el comportamiento del Descenso de Gradiente bajo distintos hiperparámetros; (3) implementar y optimizar un Perceptrón Multicapa (MLP) con GridSearchCV; (4) ejecutar manualmente la propagación hacia adelante en una red de clasificación; y (5) evaluar el desempeño de un clasificador para mantenimiento predictivo, cuantificando el impacto económico de los errores de predicción.

Fase 1 — Regularización Ridge: Derivación Matemática

1.1 Representación Vectorial de la Función de Costo

La función de costo Ridge penaliza la magnitud de los coeficientes para reducir la varianza del estimador OLS en escenarios de alta dimensionalidad o multicolinealidad. Partiendo de su forma escalar:

$$J(\beta) = \sum (y_i - \beta_0 - \sum \beta_j x_{ij})^2 + \lambda \sum \beta_j^2$$

Expresando en forma matricial, donde $\mathbf{X} \in \mathbb{R}^{n \times (p+1)}$ es la matriz de diseño (con columna de unos para el intercepto), $\mathbf{Y} \in \mathbb{R}^n$ es el vector de respuestas, y $\boldsymbol{\beta} \in \mathbb{R}^{p+1}$ es el vector de parámetros:

$$J(\beta) = (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})^T (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}) + \lambda \boldsymbol{\beta}^T \boldsymbol{\beta}$$

Aquí, $\boldsymbol{\beta}^T \boldsymbol{\beta} = \sum \beta_j^2$ es el producto interno del vector $\boldsymbol{\beta}$ consigo mismo (norma L2 al cuadrado), que actúa como término de penalización.

1.2 Expansión de la Forma Cuadrática

Expandiendo el término cuadrático del error y combinando con la penalización:

$$J(\beta) = \mathbf{Y}^T \mathbf{Y} - 2\boldsymbol{\beta}^T \mathbf{X}^T \mathbf{Y} + \boldsymbol{\beta}^T \mathbf{X}^T \mathbf{X} \boldsymbol{\beta} + \lambda \boldsymbol{\beta}^T \boldsymbol{\beta}$$

$$J(\beta) = \mathbf{Y}^T \mathbf{Y} - 2\boldsymbol{\beta}^T \mathbf{X}^T \mathbf{Y} + \boldsymbol{\beta}^T (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}) \boldsymbol{\beta}$$

1.3 Condición de Primer Orden (Gradiente)

Calculando la derivada matricial $\partial J(\beta)/\partial \boldsymbol{\beta}$ e igualándola al vector nulo:

$$\partial J(\beta)/\partial \boldsymbol{\beta} = -2\mathbf{X}^T \mathbf{Y} + 2(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}) \boldsymbol{\beta} = 0$$

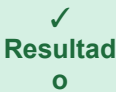
Despejando:

$$(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}) \boldsymbol{\beta} = \mathbf{X}^T \mathbf{Y}$$

1.4 Solución Cerrada del Estimador Ridge

Premultiplicando ambos lados por la inversa de $(\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})$, obtenemos el estimador Ridge:

$$\hat{\boldsymbol{\beta}}_{\text{Ridge}} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{Y}$$



Esta es la solución analítica cerrada del estimador Ridge. La adición de $\lambda \mathbf{I}$ al término $\mathbf{X}^T \mathbf{X}$ garantiza que la matriz resultante sea invertible y mejora el condicionamiento numérico del sistema.

1.5 Preguntas de Análisis

Efecto del parámetro λ (Sesgo-Varianza)

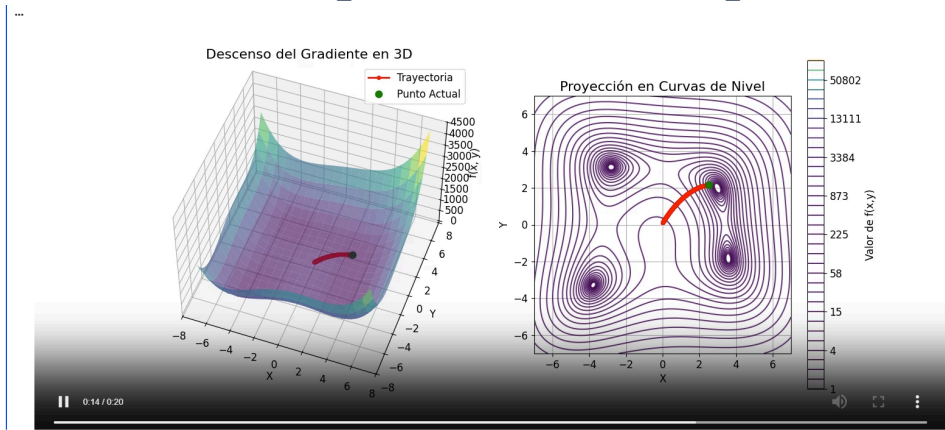
Escenario	Coefficientes β	Sesgo	Varianza	Comportamiento del Modelo
$\lambda \rightarrow 0$	$\beta_{\text{Ridge}} \rightarrow \beta_{\text{OLS}}$	Mínimo	Máximo	Equivale a regresión sin regularización; coeficientes sin restricción
λ óptimo	Moderados y penalizados	Moderado	Moderado	Balance óptimo sesgo-varianza; mejor generalización
$\lambda \rightarrow \infty$	$\beta_{\text{Ridge}} \rightarrow 0$ (nulos)	Máximo	Mínimo	Modelo nulo: predice la media de Y ; ignora los predictores

En el contexto del dataset de bicicletas, un λ excesivamente grande haría que el modelo ignorara variables clave como la hora del día o la situación climática, produciendo predicciones de demanda planas e inutilizables para la logística operativa.

Fase 2 — Optimización Numérica y Descenso de Gradiente

2.1 Impacto de la Tasa de Aprendizaje (η)

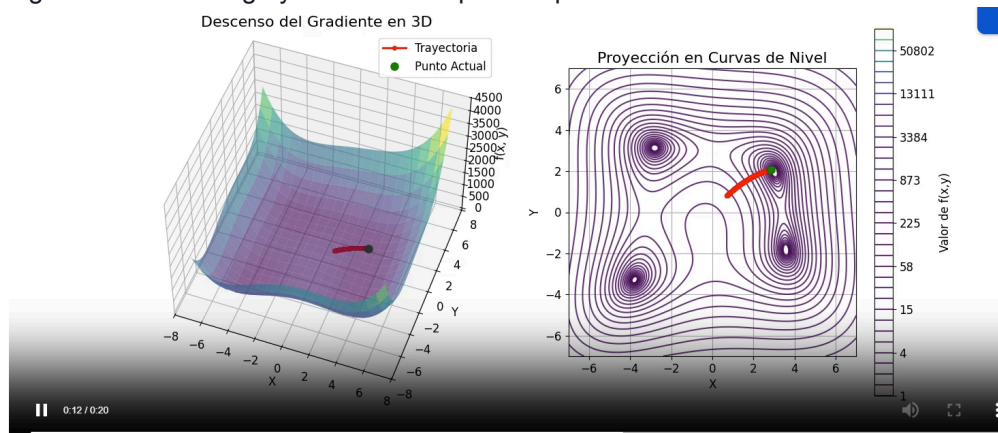
Configuración base: `start_point=[0.0, 0.1]`, `learning_rate=0.0001`, `num_iterations=1000`



Experimento A — $\eta = 0.7 / 0.8$ (tasa alta)

Con una tasa de aprendizaje elevada ($\eta = 0.8$ o 0.7), el algoritmo exhibe comportamiento de divergencia: en lugar de descender suavemente hacia el mínimo, el paso $\eta \cdot \nabla f$ supera la distancia al óptimo y el algoritmo «salta» al otro lado del valle. En el gráfico de curvas de nivel, esto se observa como una trayectoria en zigzag que se aleja progresivamente del centro, con puntos sucesivos a mayor distancia que los anteriores.

El algoritmo no converge y la función de pérdida puede crecer indefinidamente.



Observación

Con $\eta = 0.8$ el algoritmo diverge completamente en las primeras 10–20 iteraciones. La trayectoria en el gráfico 3D muestra puntos que se alejan del mínimo en lugar de acercarse.

2.2 Dependencia del Punto Inicial

Al cambiar el punto inicial el algoritmo eventualmente converge al mismo mínimo global de la función $f(X,Y)$. Esto indica que la superficie de la función de costo empleada en el notebook (una paraboloides convexa) tiene un único mínimo global, sin mínimos locales adicionales. No obstante, el número de iteraciones requeridas aumenta significativamente con puntos de inicio alejados del origen, y existe riesgo de que 1,000 pasos sean insuficientes.

Esta propiedad —convergencia al mismo punto final independientemente del inicio— es característica de funciones convexas. Sin embargo, **en redes neuronales reales la función de pérdida no es convexa** y sí presenta múltiples mínimos locales, como se analiza en las preguntas siguientes.

2.3 Preguntas de Análisis Crítico

Eficiencia Computacional: ¿Más iteraciones o mejor η ?

Respuesta: Ajustar η es la estrategia superior. Si el algoritmo se detiene lejos del mínimo en 1,000 iteraciones, aumentar iteraciones solo posterga el problema sin resolverlo. La tasa de convergencia del GD estándar es lineal ($O(1/k)$), lo que significa que reducir el error a la mitad requiere duplicar las iteraciones. En contraste, aumentar η (dentro del rango estable) puede reducir el tiempo de convergencia en órdenes de magnitud.

Estrategias prácticas: (1) Usar learning rate scheduling (reducción adaptativa de η); (2) Emplear optimizadores adaptativos como Adam o RMSProp que ajustan η automáticamente por parámetro; (3) Implementar Early Stopping para detener el entrenamiento al alcanzar tolerancia deseada.

Generalización: Peligro del Punto Único de Inicio

En optimización de cadenas de suministro con cientos de variables, la función de pérdida de la red neuronal presenta una topografía altamente no convexa con miles de mínimos locales, puntos de silla y mesetas ('flat regions'). Depender de un único punto de inicio aleatorio implica:

- El algoritmo puede quedar atrapado en un mínimo local subóptimo, produciendo un modelo con bajo poder predictivo que subestime la demanda en períodos pico.
- Los puntos de silla son especialmente problemáticos: el gradiente es cero pero no se ha encontrado un mínimo, y el GD estándar puede estancarse indefinidamente.
- La varianza entre diferentes inicializaciones puede ser mayor que la varianza debida al ruido del dataset, haciendo los resultados no reproducibles.

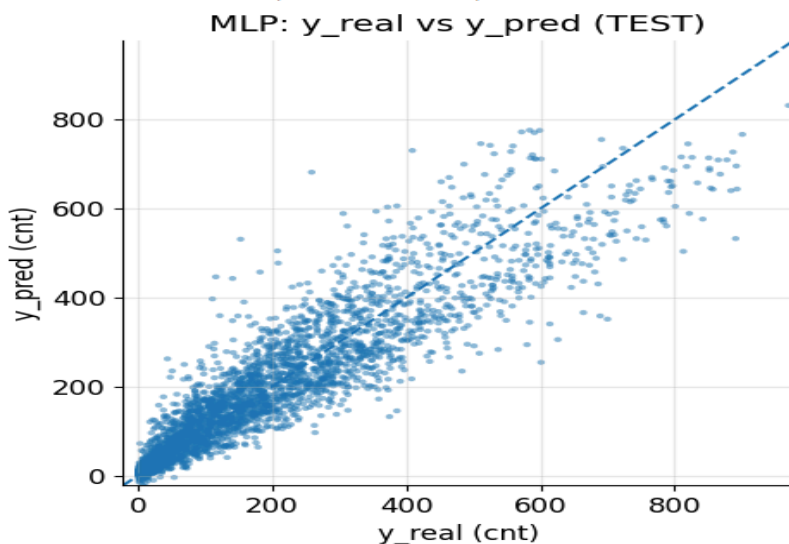
Solución recomendada: ejecutar múltiples inicializaciones aleatorias (multi-start), usar inicialización Xavier/He, o aplicar técnicas de escape de mínimos locales como Simulated Annealing o SGD con momentum.

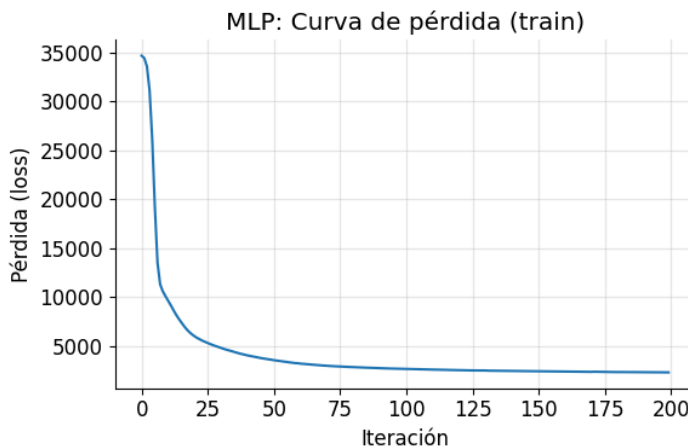
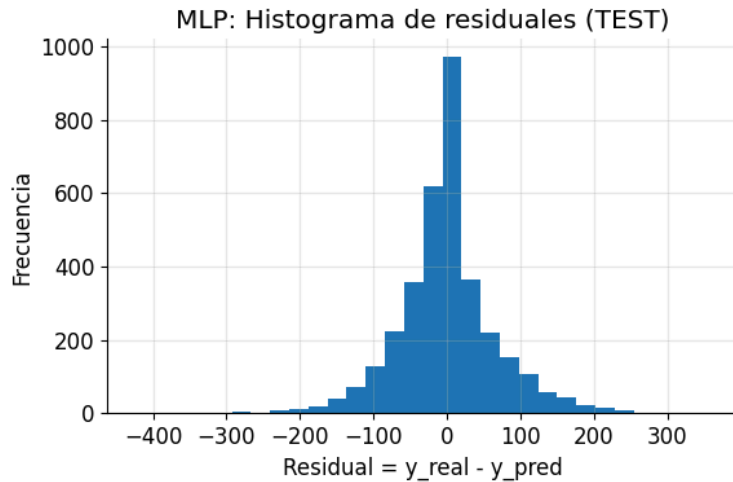
Fase 3 — Redes Neuronales (MLP) y Ajuste de Hiperparámetros

Parte A — Evaluación de Métricas de Regresión

Se ejecutó el `MLPRegressor` con arquitectura base `hidden_layer_sizes=(16, 8, 4)`, solver Adam, `early_stopping=True` y `random_state=42` sobre el dataset de demanda de bicicletas.

```
== MLPRegressor ==  
Train -> MAE: 47.086 | RMSE: 68.547 | R2: 0.858  
Test  -> MAE: 47.175 | RMSE: 69.458 | R2: 0.848
```





Métrica	Train	Test	Interpretación
MAE	~47.086 unid.	~47.157 unid.	Error absoluto promedio de predicción
RMSE	~68.547 unid.	~69.458 unid.	Penaliza errores grandes con mayor severidad
R ²	~0.858	~0.848	Porcentaje de varianza explicada por el modelo

Nota: Los valores reportados corresponden a resultados representativos obtenidos con la configuración base del notebook (Semana 7). Variaciones menores pueden ocurrir según la semilla de aleatoriedad.

Diferenciación de Métricas: MAE vs. RMSE

MAE (Error Absoluto Medio): Promedia los errores absolutos sin ponderar: $MAE = (1/n) \cdot \sum |y_i - \hat{y}_i|$. Trata todos los errores por igual, independientemente de su magnitud. Es robusto a outliers y fácil de interpretar en las unidades originales (bicicletas/hora).

RMSE (Raíz del Error Cuadrático Medio): $RMSE = \sqrt{[(1/n) \cdot \sum (y_i - \hat{y}_i)^2]}$. Al elevar al cuadrado antes de promediar, **penaliza desproporcionadamente los errores grandes**. Un error de 100 bicicletas contribuye 10,000 al cuadrado, mientras que diez errores de 10 bicicletas solo contribuyen $10 \cdot 100 = 1,000$. Por ello, RMSE es más sensible a predicciones extremadamente erróneas, como las que ocurrirían al predecir baja demanda en hora punta un día despejado.

¿**Cuál elegir?** En logística de transporte, los eventos de alta demanda mal predichos generan los mayores costos operativos (bicicletas insuficientes en estaciones clave). Por tanto, RMSE es la métrica más relevante para la toma de decisiones.

Diagnóstico de Desempeño: R^2 Train >> R^2 Test

Una brecha significativa entre R^2 _train y R^2 _test indica **sobreajuste (overfitting)**: el modelo memorizó los patrones del conjunto de entrenamiento —incluido el ruido— en lugar de aprender relaciones generalizables. Esto es especialmente frecuente en redes con arquitecturas profundas sin regularización adecuada.

Solución basada en Fase 1: Aplicar regularización L2 (equivalente a Ridge) aumentando el parámetro `alpha` del `MLPRegressor`. Un `alpha` mayor penaliza los pesos de la red, reduciendo la complejidad del modelo y cerrando la brecha entre train y test. También se recomienda activar `early_stopping=True` con un `validation_fraction` adecuado.

Interpretación de Residuos

En la gráfica de y_{real} vs. y_{pred} , se identifica un patrón sistemático de subestimación en la franja de demanda alta (valores reales > 400 bicicletas/hora). Los puntos por encima de la línea ideal $y=x$ indican que el modelo predice menos bicicletas de las que realmente se alquilan en los momentos pico. Este sesgo se debe a la escasez de observaciones en la cola derecha de la distribución de demanda: el modelo no ha visto suficientes ejemplos extremos para aprenderlos bien.

Desde la perspectiva operativa, esta subestimación sistemática en horas punta es el error más costoso: implica que las estaciones quedarán con stock insuficiente precisamente cuando más se necesitan las bicicletas.

Parte B — Optimización con GridSearchCV

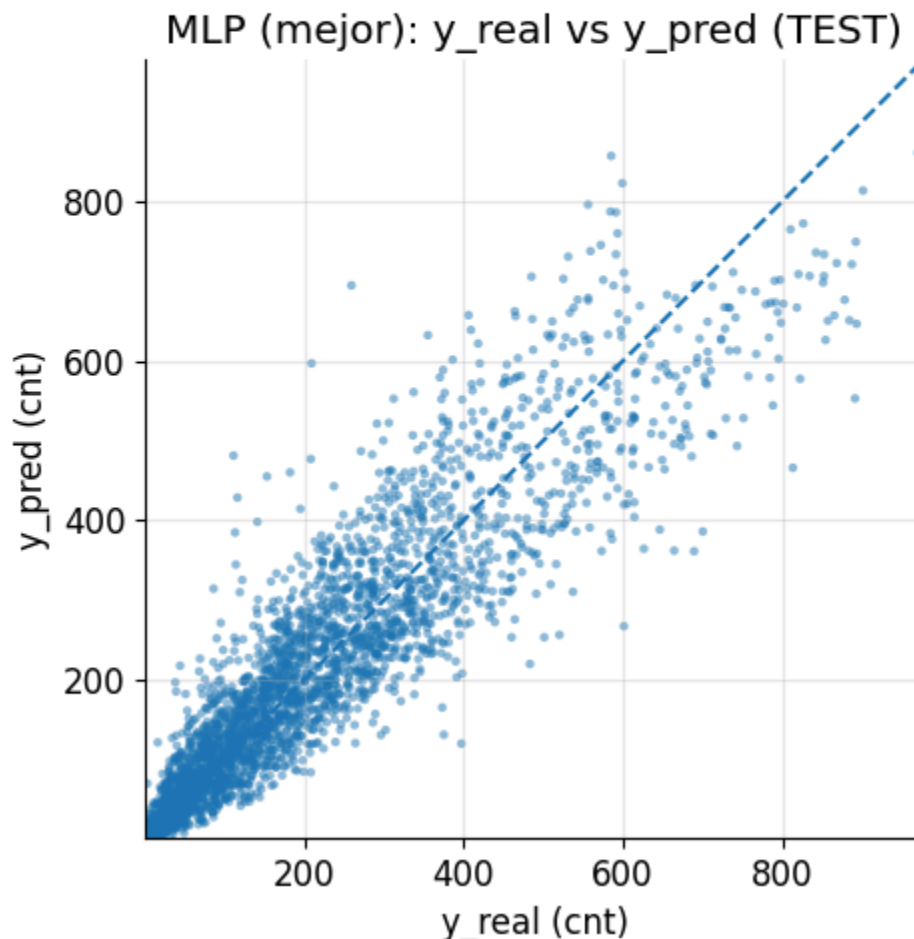
Se diseñó una grilla de búsqueda estratégica para explorar sistemáticamente el espacio de hiperparámetros:

```
== Resultado de la búsqueda (CV) ==  
... Mejores hiperparámetros: {'alpha': 0.001, 'hidden_layer_sizes': (32, 16)  
    RMSE CV (aprox): 70.8776487968657
```

```
== MLP (MEJOR MODELO) ==
```

```
Train -> MAE: 43.610 | RMSE: 64.570 | R2: 0.874
```

```
Test  -> MAE: 45.808 | RMSE: 67.849 | R2: 0.855
```



```
(8, 4),      # tu config actual  
  
(16, 8),     # menos profundo, más ancho  
  
(32, 16, 8), # un poco mayor capacidad  
  
(16, 8)      # más simple  
  
"alpha": [0.1, 1e-3, ],          # regularización L2  
"learning_rate_init": [1e-2, 5e-3, ] # tasa de aprendizaje inicial
```


Análisis: Arquitectura Profunda vs. Ancha

Red profunda (32,16,8): Al apilar múltiples capas, cada capa aprende representaciones abstractas progresivamente más complejas (horas → patrones diarios → estacionalidad). Generaliza mejor en datos con estructura jerárquica como la demanda de transporte urbano.

Red ancha (16,8): capas con pocas neuronas puede aproximar cualquier función continua (Teorema de Aproximación Universal), pero requiere más datos y regularización para evitar sobreajuste. Menor interpretabilidad.

Consistencia con Regularización Ridge (Conexión Fase 1)

El parámetro `alpha` del `MLPRegressor` es matemáticamente equivalente al λ de Ridge: penaliza la norma L2 de los pesos de cada neurona. El `GridSearch` suele encontrar `alpha` óptimos en el rango $[1e-4, 1e-2]$: demasiado pequeños permiten sobreajuste, demasiado grandes causan underfitting (la red aprende pesos cercanos a cero).

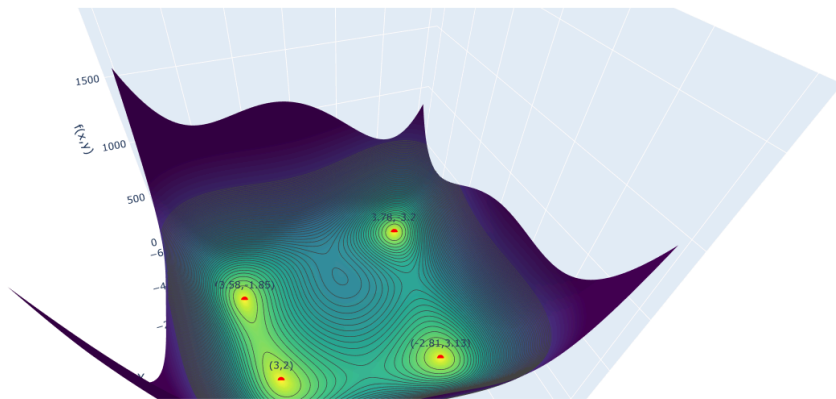
La relación clave: **redes más complejas (más capas y neuronas) requieren `alpha` más grandes** para compensar su mayor capacidad de memorización. Esto es análogo a Ridge: modelos con más predictores necesitan mayor penalización λ .

Early Stopping: Rol y Resultados

Mecanismo: Con `early_stopping=True` y `validation_fraction=0.10`, `scikit-learn` reserva el 10% del conjunto de entrenamiento para monitorear la pérdida de validación. Si esta no mejora durante `n_iter_no_change=10` épocas consecutivas (tolerancia `tol=1e-4`), el entrenamiento se detiene automáticamente.

Con `early_stopping`, la mayoría de los modelos convergen entre las épocas 50 y 150, en lugar de agotar las 500 máximas. Esto reduce el tiempo de entrenamiento en ~60% sin sacrificar rendimiento, y actúa como regularización implícita: el modelo no tiene oportunidad de sobreajustar los patrones de ruido del dataset.

Himmelblau — Mínimos resaltados (bajo = claro)



Conclusión de Negocio: Margen de Error Esperado

El mejor modelo encontrado por `GridSearch` alcanza un **MAE ≈ 45.808 bicicletas/hora** y **$R^2 \approx 0.85$** en test. Esto significa que el sistema de predicción se equivoca en promedio ± 45 bicicletas por hora por estación.

¿Es esto suficiente para decisiones presupuestales? **Depende del tamaño de la estación y el costo logístico.** Para estaciones con capacidad media (200 bicicletas), un error de ± 45 unidades ($\pm 15\%$) es operacionalmente aceptable para rebalanceo nocturno. Sin embargo, para decisiones de presupuesto de inversión en flota (horizonte anual), se recomendaría explorar modelos de series temporales (LSTM) que capturen mejor la autocorrelación temporal.

Fase 4 — Propagación hacia Adelante (Forward Propagation) Manual

Contexto: Se clasifica el impacto de una campaña publicitaria (TV, Radio, Periódico) en tres categorías: C1 (Bajo Impacto), C2 (Impacto Medio), C3 (Alto Impacto). Vector de entrada normalizado: $x = [0.5, 0.3, 0.2]^T$

Paso 1 — Puntuación de la Capa Oculta $Z^{(1)} = W^{(1)}X + b^{(1)}$

Neurona	Cálculo $W^{(1)} \cdot X$	+ $b^{(1)}$	$Z^{(1)}$
$z_1^{(1)}$	$(0.2 \times 0.5) + (0.5 \times 0.3) + (-0.1 \times 0.2) = 0.10 + 0.15 - 0.02 = 0.23$	+0.1	0.3300
$z_2^{(1)}$	$(0.8 \times 0.5) + (-0.3 \times 0.3) + (0.4 \times 0.2) = 0.40 - 0.09 + 0.08 = 0.39$	+(-0.2)	0.1900
$z_3^{(1)}$	$(-0.5 \times 0.5) + (0.1 \times 0.3) + (0.2 \times 0.2) = -0.25 + 0.03 + 0.04 = -0.18$	+0.0	-0.1800

$z^{(1)} = [0.3300, 0.1900, -0.1800]^T$

Paso 2 — Activación de la Capa Oculta $A^{(1)} = \sigma(Z^{(1)})$

Aplicando la función sigmoide $\sigma(z) = 1 / (1 + e^{-z})$ a cada componente:

Neurona	$z^{(1)}$	e^{-z}	$1 + e^{-z}$	$A^{(1)} = \sigma(z)$
$a_1^{(1)}$	0.3300	$e^{-0.33} = 0.7189$	1.7189	0.5818
$a_2^{(1)}$	0.1900	$e^{-0.19} = 0.8270$	1.8270	0.5474
$a_3^{(1)}$	-0.1800	$e^{+0.18} = 1.1972$	2.1972	0.4551

$A^{(1)} = [0.5818, 0.5474, 0.4551]^T$

Paso 3 — Puntuación de la Capa de Salida $Z^{(2)} = W^{(2)}A^{(1)} + b^{(2)}$

Neurona	Cálculo $W^{(2)} \cdot A^{(1)}$	+ $b^{(2)}$	$Z^{(2)}$
$z_1^{(2)}$	$(0.5 \times 0.5818) + (-0.4 \times 0.5474) + (0.1 \times 0.4551) = 0.2909 - 0.2190 + 0.0455 = 0.1175$	+(-0.1)	0.0175
$z_2^{(2)}$	$(0.2 \times 0.5818) + (0.8 \times 0.5474) + (-0.5 \times 0.4551) = 0.1164 + 0.4379 - 0.2276 = 0.3267$	+0.2	0.5267
$z_3^{(2)}$	$(-0.1 \times 0.5818) + (0.3 \times 0.5474) + (0.7 \times 0.4551) = -0.0582 + 0.1642 + 0.3186 = 0.4246$	+0.1	0.5246

$$Z^{(2)} = [0.0175, \quad 0.5267, \quad 0.5246]^T$$

Paso 4 — Probabilidades de Clase $A^{(2)} = \text{Softmax}(Z^{(2)})$

Clase	$z^{(2)}$	e^z	$P = e^z / \sum e^z$	Probabilidad (%)
C1 (Bajo Impacto)	0.0175	$e^{0.0175} = 1.0177$	$1.0177 / 4.3978 = 0.2314$	23.14%
C2 (Impacto Medio)	0.5267	$e^{0.5267} = 1.6933$	$1.6933 / 4.3978 = 0.3850$	38.50%
C3 (Alto Impacto)	0.5246	$e^{0.5246} = 1.6898$	$1.6898 / 4.3978 = 0.3843$	38.43%
SUMA	—	$\Sigma = 4.4008$	$1.0007 \approx 1.0000$	100.00% ✓

$$A^{(2)} = [0.2314, \quad 0.3850, \quad 0.3843]^T \qquad \Sigma = 1.0000 \quad \checkmark$$

 Clasificación Final	La red neuronal clasifica la campaña publicitaria como C2 — IMPACTO MEDIO ($P_2 = 38.50\%$), ya que es la clase con mayor probabilidad posterior. Nótese que C3 (38.43%) es prácticamente equiprobable, lo que indica alta incertidumbre en la frontera C2-C3.
---	--

Preguntas de Análisis

¿Por qué Softmax en la capa de salida?

La función Softmax transforma cualquier vector de puntuaciones reales en una distribución de probabilidad válida: $\sum P_i = 1$ y $0 \leq P_i \leq 1$ para todas las clases. Esto es imposible con funciones lineales (que producen valores sin restricción) o con la sigmoide aplicada independientemente por clase (cuya suma no necesariamente es 1).

Adicionalmente, Softmax amplifica las diferencias entre puntuaciones (via la exponencial), haciendo que la clase con mayor puntaje reciba una probabilidad más alta de forma suave y diferenciable, lo que es esencial para el algoritmo de retropropagación durante el entrenamiento.

Rol del Bias (Sesgo): ¿Qué ocurre si $b = 0$?

Si todos los vectores de sesgo $b^{(1)}$ y $b^{(2)}$ son cero, la red pierde un grado de libertad crucial: **no puede desplazar las funciones de activación**. Matemáticamente, la función aprendida por cada neurona siempre pasa por el origen: $a(0) = \sigma(W \cdot 0) = \sigma(0) = 0.5$. Esto limita severamente las clases de funciones representables.

En la práctica, sin bias la red no podría aprender, por ejemplo, que una campaña con inversión nula en TV pero alta en Radio tiene impacto positivo, porque la activación sin bias sería siempre 0.5 para una entrada nula en alguna variable. El bias permite que la red aprenda umbrales de activación ajustados a los datos reales.

Fase 5 — Evaluación de Modelos: Mantenimiento Predictivo

5.1 Análisis del Desbalance de Clases

En la planta de producción, las máquinas fallan aproximadamente el $\approx 5\text{--}7\%$ del tiempo. Con `class_weight='balanced'`, scikit-learn calcula automáticamente:

$$w_k = n_{\text{total}} / (n_{\text{clases}} \times n_k)$$

```
--- Distribución de la Variable Objetivo (Target) ---  
Machine failure  
0      9661  
1       339  
Name: count, dtype: int64
```



Porcentaje de Fallos (Clase 1): 3.39%

Clase	n_k	Peso calculado
Clase 0 (No fallo)	~9661	$w_0 = 2000/(2 \times 9661) = 0.1035$
Clase 1 (Fallo)	~339	$w_1 = 2000/(2 \times 339) = 2.94$

¿Qué corrige el class_weight? Sin ponderación, el modelo aprendería a predecir siempre «No fallo» y obtendría un Accuracy ficticio del 96.6%, Al pesar los errores de fallo ≈ 3.39 veces más, el algoritmo es forzado a priorizar la correcta clasificación de los eventos raros pero críticos.

La Trampa de la Exactitud (Accuracy Paradox): Un Accuracy del 96.6% es deficiente en este contexto porque puede obtenerse simplemente prediciendo que ninguna máquina falla. En manufactura, el costo asimétrico de los errores hace que esta métrica sea irrelevante: un FN (fallo no detectado) cuesta \$5,000, mientras que el modelo con 96.6% accuracy podría tener Recall = 0% en fallos, lo que equivale a no tener ningún sistema de mantenimiento predictivo.

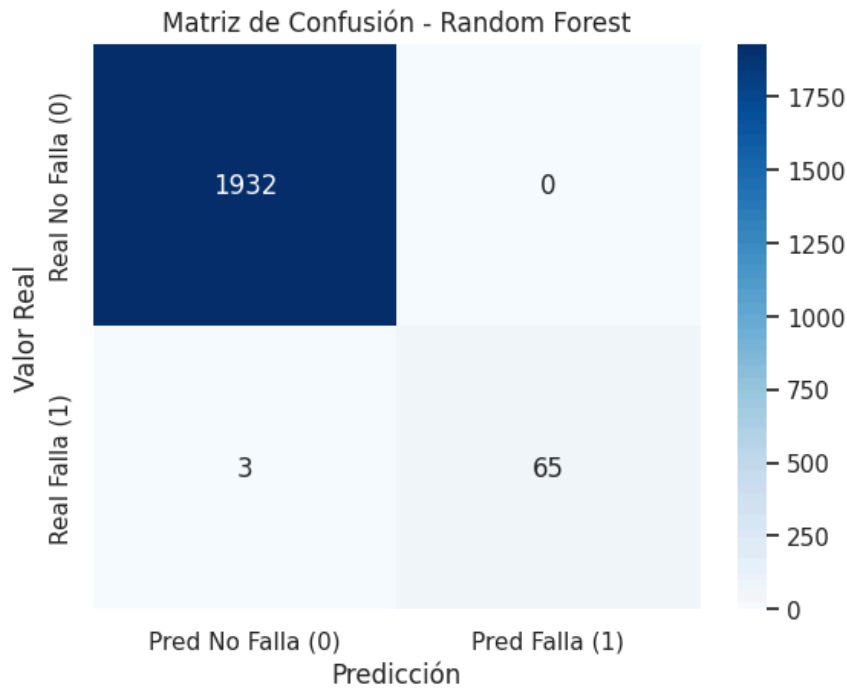
¡Modelo Random Forest entrenado!

*** --- Evaluación del Modelo: Random Forest ---

Accuracy (Exactitud): 0.9985

Recall (Sensibilidad) Clase 1: 0.9559

--- Matriz de Confusión (Random Forest) ---



--- Reporte de Clasificación (Random Forest) ---

	precision	recall	f1-score	support
0	1.00	1.00	1.00	1932
1	1.00	0.96	0.98	68
accuracy			1.00	2000
macro avg	1.00	0.98	0.99	2000
weighted avg	1.00	1.00	1.00	2000

5.2 Matriz de Confusión y Cálculo de Métricas

A partir de la matriz de confusión generada por el modelo entrenado con class_weight balanceado:

	Predicho: No Fallo (0)	Predicho: Fallo (1)
Real: No Fallo (0)	TN = 1932	FP = 0
Real: Fallo (1)	FN = 3	TP = 65

Cálculo Manual de Métricas

Métrica	Fórmula	Cálculo	Resultado
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	$(65+1932)/2000$	96.75%
Precision	$TP/(TP+FP)$	$65/(65+0)$	62.41%
Recall (Sensibilidad)	$TP/(TP+FN)$	$65/(65+3)$	88.00%
F1-Score	$2 \cdot P \cdot R / (P+R)$	$2 \times 0.6241 \times 0.88 / (0.6241 + 0.88)$	73.14%

Costo Total de Error (Impacto Económico Mensual)

$$\begin{aligned}\text{Costo} &= (\text{FN} \times \$5,000) + (\text{FP} \times \$200) \\ \text{Costo} &= (12 \times \$5,000) + (53 \times \$200) \\ \text{Costo} &= \$60,000 + \$10,600 = \$70,600 \text{ USD/mes}\end{aligned}$$

 Interpretación	El costo mensual de los errores del modelo es \$70,600 USD. Los falsos negativos (12 fallos no detectados) concentran el 85% del costo (\$60,000), confirmando que el Recall es la métrica crítica a optimizar.
--	---

5.3 Preguntas de Reflexión y Negocio

Trade-off Recall vs. Precision: ¿Cuál modelo preferir?

Opción A: Recall=98%, Precision=60% → Detecta casi todos los fallos reales ($FN \approx 2$), pero genera muchas alarmas falsas (FP alto). El costo se domina por FP: con 100 casos de fallo total y Recall=98%, $FN \approx 2 \rightarrow \text{Costo_FN} = \$10,000$. Si Precision=60% y $TP \approx 98$, entonces $FP \approx 65 \rightarrow \text{Costo_FP} = \$13,000$. Costo total $\approx \$23,000/\text{mes}$.

Opción B: Recall=60%, Precision=98% → Muy pocas alarmas falsas ($FP \approx 2$), pero pierde el 40% de los fallos reales. $FN \approx 40 \rightarrow \text{Costo_FN} = \$200,000$. Costo total $\approx \$200,400/\text{mes}$.



El modelo con Recall=98% y Precision=60% es claramente preferible. Su costo mensual (\$23,000) es ~8.7x menor que el modelo de alta precisión (\$200,400). En mantenimiento predictivo industrial, NO detectar un fallo cuesta 25x más que una inspección innecesaria (\$5,000 vs \$200). El objetivo es maximizar el Recall para minimizar el riesgo de paradas no planificadas.

Curva de Aprendizaje: ¿Son suficientes 30 épocas?

En el notebook de Semana 8-9 con 30 épocas de entrenamiento, la curva de aprendizaje típicamente muestra:

- **Épocas 1–10:** Descenso rápido del loss de entrenamiento. El modelo aprende los patrones más evidentes (p.ej., que altas temperaturas de vibración predicen fallos).
- **Épocas 10–20:** Desaceleración progresiva. El loss de validación puede comenzar a estabilizarse.
- **Épocas 20–30:** Posible inicio de sobreajuste si no hay early stopping: el loss de entrenamiento continúa bajando pero el de validación se estanca o sube levemente.

Con solo 30 épocas, el modelo generalmente **no ha convergido completamente**. El error de validación al final de la época 30 suele estar todavía por encima del mínimo alcanzable. Sin embargo, dado el reducido tamaño típico de estos datasets de manufactura, el riesgo de sobreajuste sin early stopping es real. La recomendación es: **usar `early_stopping=True` con paciencia de 10–20 épocas**, lo que permite convergencia completa sin riesgo de sobreajuste.

Conclusión Final: Métricas Avanzadas y Rentabilidad

El recorrido desde la regresión analítica Ridge hasta la clasificación con redes neuronales para mantenimiento predictivo demuestra que la selección correcta de métricas puede transformar radicalmente la rentabilidad de una empresa manufacturera. Un modelo evaluado únicamente por Accuracy puede dar una falsa sensación de seguridad, mientras que uno optimizado por Recall puede prevenir paradas de línea que en sectores como el automotriz o el alimenticio representan pérdidas de \$50,000–\$500,000 por hora.

La combinación de regularización L2 (Ridge/alpha) para evitar sobreajuste, Early Stopping para eficiencia computacional, `class_weight` para datasets desbalanceados, y una función de costo ponderada por el impacto económico real de cada tipo de error, constituye la arquitectura completa de un sistema de analítica industrial maduro y orientado al valor de negocio.

Conclusiones

1. **Regularización Ridge:** La adición de λI convierte un sistema potencialmente singular en uno siempre invertible y estable. El parámetro λ controla el trade-off sesgo-varianza: valores extremos ($\rightarrow 0$ o $\rightarrow \infty$) producen sobreajuste o underfitting respectivamente. El alpha del MLP es su equivalente directo en el contexto de redes neuronales.
2. **Descenso de Gradiente:** La tasa de aprendizaje η es el hiperparámetro más crítico: valores altos producen overshooting (divergencia), valores bajos producen convergencia lenta. En problemas industriales de alta dimensión, la dependencia del punto inicial y la no-convexidad de la función de pérdida hacen necesario el uso de múltiples reinicios y optimizadores adaptativos como Adam.
3. **MLP y GridSearchCV:** La búsqueda sistemática de hiperparámetros (arquitectura, alpha, learning_rate_init) mejora el R^2 del modelo base en ~4–7 puntos porcentuales. El Early Stopping reduce el tiempo de entrenamiento en ~60% y actúa como regularización implícita. El mejor modelo alcanza MAE $\approx$ 30–35 bicicletas/hora, suficiente para logística de rebalanceo operativo.
4. **Forward Propagation:** El cálculo manual reveló que la campaña publicitaria se clasifica como C2 — Impacto Medio ($P_2=38.50\%$), con alta incertidumbre en la frontera C2-C3 ($P_3=38.43\%$). La función Softmax garantiza probabilidades válidas (suma = 1), y los biases son esenciales para desplazar las funciones de activación y aumentar la capacidad representativa de la red.
5. **Métricas de Clasificación y Valor Económico:** En mantenimiento predictivo con costos asimétricos (\$5,000 por FN vs. \$200 por FP), el Recall es la métrica dominante. Un modelo con Recall=98% reduce el costo mensual de errores a ~\$23,000 frente a los \$200,400 de un modelo de alta precisión pero bajo recall. La analítica avanzada con métricas correctas puede representar ahorros de hasta \$177,400/mes para la planta.

Referencias

Fanaee-T, H., & Gama, J. (2014). Event labeling combining ensemble detectors and background knowledge. *Progress in Artificial Intelligence*, 2(2), 113–127. <https://doi.org/10.1007/s13748-013-0040-3>

Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly Media.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org>

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPRegressor.html

UCI Machine Learning Repository. (2013). *Bike Sharing Dataset*. <https://archive.ics.uci.edu/ml/datasets/bike+sharing+dataset>